
PerceptionLab

Design Patterns & Conventions fichiers/dossiers

Architecture pass - v1.0 - 2026-06-15

Objectif : définir les patterns obligatoires, les anti-patterns interdits, les règles de dépendance, les conventions de nommage et la structure de dossiers du projet PerceptionLab.

Ce document est volontairement normatif. Son rôle est d'empêcher la dérive architecturale : chaque pattern doit protéger une frontière, un invariant métier, un contrat typé ou un point d'évolution futur.

Projet	PerceptionLab
Périmètre	API Rust, worker Python/PyTorch, PostgreSQL, storage objet, QA BDD
Style d'architecture	Architecture hexagonale, use cases, domaine fortement typé
But	Construire une plateforme ML infrastructure maintenable, pas une simple démo notebook

Table des matières

1. Positionnement de la passe Design Patterns	4
Règle de décision	4
2. Principe d'architecture	4
Règle de dépendance	4
Dépendances interdites	4
3. Carte des patterns obligatoires	5
4. Catalogue détaillé des patterns	5
4.1 Hexagonal Architecture / Ports & Adapters	5
4.2 Use Case Pattern	5
4.3 Newtype Pattern	6
4.4 Value Object Pattern	6
4.5 Repository Pattern	6
4.6 Unit of Work / Transaction Boundary	6
4.7 Strategy Pattern	7
4.8 State Machine Pattern	7
4.9 DTO + Mapper Pattern	7
4.10 Adapter Pattern	7
4.11 Factory / Builder Pattern	8
4.12 Error Mapping Pattern	8
4.13 Lightweight Event Log	8
5. Patterns par module produit	8
6. State machines obligatoires	9
Training Job	9
Model	9
Export	9
Dataset	9
7. Patterns interdits et anti-patterns	9
8. Conventions fichiers/dossiers Rust	10
Règles Rust	10
9. Conventions fichiers/dossiers Python worker	11
Règles Python	11
10. Conventions QA, contracts, docs et scripts	12
QA	12
Contracts	12
Docs	12
Scripts	12
11. Conventions de nommage	13
Règle des fichiers interdits	13
12. Exemples Rust	13

Newtype ID	13
Value Object avec invariant	13
Repository port	14
Use case	14
State transition	14
13. Exemples Python worker	14
Pydantic strict contract	14
Protocol Strategy	14
FakeTrainer pour CI	15
Application service	15
14. Tests attendus par pattern	15
Mapping tests vers dossiers	15
15. Règles d'import et dépendances	16
Contrôle PR	16
16. Checklist de review architecture	16
17. Roadmap d'implémentation	16
Première victoire attendue	17
18. Synthèse	17

1. Positionnement de la passe Design Patterns

Cette passe transforme les choix d'architecture en règles pratiques d'organisation du code. Les patterns ne sont pas décoratifs : ils existent pour rendre le code plus lisible, plus testable et plus difficile à casser.

Le produit cible est une plateforme ML infrastructure. Il faut donc articuler une API Rust robuste, une couche application lisible, une persistance PostgreSQL cohérente, un storage objet, un worker Python/PyTorch et une QA BDD transverse.

Règle centrale : l'arborescence doit expliquer le produit. Un reviewer doit comprendre immédiatement où vivent le domaine, les use cases, les ports, les adapters, les DTO, les contrats et les tests.

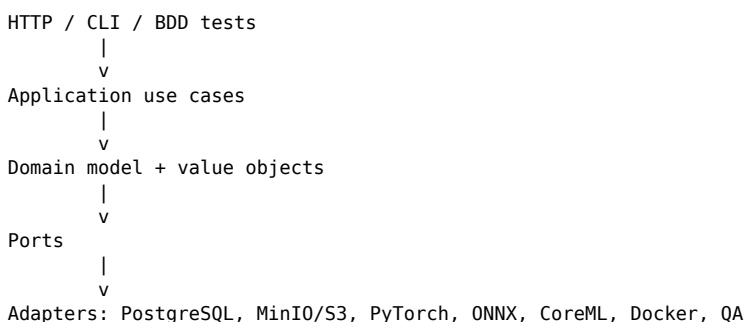
Règle de décision

- Quel invariant métier ce pattern protège-t-il ?
- Quelle dépendance technique ce pattern isole-t-il ?
- Quelle évolution future ce pattern rend-il moins coûteuse ?
- Quel test ce pattern rend-il plus facile à écrire ?
- Quelle confusion de responsabilité ce pattern évite-t-il ?

Interdiction implicite : ajouter un pattern juste parce qu'il a l'air senior. Le repo doit rester compact, lisible et pragmatique.

2. Principe d'architecture

PerceptionLab utilise une architecture hexagonale légère, aussi appelée Ports & Adapters. Le cœur du projet est le domaine. Les composants techniques tournent autour du domaine et ne doivent pas le contaminer.



Règle de dépendance

```

http -> app -> domain
infra -> app -> domain
worker app -> worker domain/contracts
qa -> public API + test DB/storage helpers

```

domain -> rien de technique

Dépendances interdites

```

domain -> sqlx
domain -> axum
domain -> MinIO / S3
domain -> PyTorch
app -> axum extractors
app -> raw SQL rows
http handler -> SQL query
worker repository -> torch

```

3. Carte des patterns obligatoires

Pattern	Rôle dans PerceptionLab	Couche principale
Hexagonal Architecture	Séparer domaine, application et adapters techniques	Global
Use Case	Représenter chaque intention produit	app
Newtype	Éviter les confusions d'IDs et de primitives	domain
Value Object	Garantir qu'une valeur invalide ne peut pas exister	domain
Repository	Masquer PostgreSQL derrière des ports	app/infra
Unit of Work	Encadrer les transactions multi-écritures	app/infra
Strategy	Changer fake/tiny/real training, storage ou inference	worker/infra
State Machine	Protéger les statuts jobs, modèles et exports	domain
DTO + Mapper	Séparer HTTP/DB/worker du domaine	http/infra/worker
Adapter	Isoler SQLx, S3, PyTorch, ONNX, CoreML	infra/worker
Factory / Builder	Centraliser le bootstrap et la config typée	bootstrap
Error Mapping	Traduire erreurs internes en réponses publiques stables	http/worker

4. Catalogue détaillé des patterns

4.1 Hexagonal Architecture / Ports & Adapters

Dimension	Convention
Objectif	Garder le cœur métier indépendant des frameworks et systèmes externes.
Utilisation	Toute l'architecture : API Rust, PostgreSQL, storage, queue, worker, inference, QA.
Règle d'implémentation	Les ports sont définis dans la couche application. Les adapters vivent dans infra ou worker/adapters.
Vérification	Tests use case avec ports fake + tests intégration avec adapters réels.
Abus interdit	Créer des abstractions inutiles pour des détails internes sans risque d'évolution.

4.2 Use Case Pattern

Dimension	Convention
Objectif	Rendre chaque intention produit explicite.
Utilisation	CreateDataset, UploadSample, AddAnnotation, CreateDatasetVersion, CreateTrainingJob, RunInference.
Règle d'implémentation	Un fichier par use case. Le handler HTTP appelle le use case, il ne l'implémente pas.
Vérification	Tester le comportement sans HTTP et sans vraie DB quand c'est possible.
Abus interdit	Créer un service géant qui contient tous les comportements.

4.3 Newtype Pattern

Dimension	Convention
Objectif	Empêcher les confusions de primitives entre IDs métier.
Utilisation	DatasetId, SampleId, AnnotationId, DatasetVersionId, TrainingJobId, ModelId, ArtifactId.
Règle d'implémentation	Wrapper les UUID/String. Convertir uniquement aux frontières HTTP, DB et logs.
Vérification	Tester parsing, sérialisation et refus des IDs invalides.
Abus interdit	Passer des String partout sous prétexte que c'est plus rapide.

4.4 Value Object Pattern

Dimension	Convention
Objectif	Garantir les invariants sur les valeurs sensibles.
Utilisation	NormalizedBbox, ImageDimensions, Checksum, ConfidenceScore, ArtifactUri, TrainingHyperparameters.
Règle d'implémentation	Champs privés + constructeur validant qui retourne Result ou ValidationError.
Vérification	Tester les bornes : bbox hors image, dimensions nulles, confidence > 1, learning rate invalide.
Abus interdit	Rendre les champs publics puis espérer que personne ne casse l'invariant.

4.5 Repository Pattern

Dimension	Convention
Objectif	Masquer la persistance derrière des méthodes métier.
Utilisation	DatasetRepository, SampleRepository, TrainingJobRepository, ModelRepository.
Règle d'implémentation	Le trait vit dans app/ports. L'implémentation SQLx vit dans infra/postgres.
Vérification	Tests intégration PostgreSQL avec migrations réelles.
Abus interdit	Mettre du SQL dans les handlers ou les use cases.

4.6 Unit of Work / Transaction Boundary

Dimension	Convention
Objectif	Rendre atomiques les opérations multi-écritures.
Utilisation	Création version dataset, lock worker, finalisation training, création modèle + métriques.
Règle d'implémentation	Transaction explicite autour du bloc critique. Durée courte. Pas de training long dans une transaction.
Vérification	Tester rollback et absence de metadata orpheline.
Abus interdit	Cacher des transactions dans des repositories de manière opaque.

4.7 Strategy Pattern

Dimension	Convention
Objectif	Changer une implémentation sans changer le use case.
Utilisation	FakeTrainer/TinyTrainer/YoloTrainer, LocalStorage/S3Storage, TorchInference/OnnxInference.
Règle d'implémentation	Un trait Rust ou Protocol Python. Injection via bootstrap/config.
Vérification	Tests de contrat communs à chaque stratégie.
Abus interdit	Disperser des if TRAINING_MODE partout dans le code.

4.8 State Machine Pattern

Dimension	Convention
Objectif	Empêcher les transitions de statut impossibles.
Utilisation	TrainingJobStatus, ModelStatus, ExportStatus, DatasetStatus.
Règle d'implémentation	Le statut change via méthodes de domaine : start(), succeed(), fail(), promote().
Vérification	Tester transitions autorisées et interdites.
Abus interdit	Modifier status directement depuis n'importe quel adapter.

4.9 DTO + Mapper Pattern

Dimension	Convention
Objectif	Séparer contrat externe et modèle métier.
Utilisation	HTTP requests/responses, DB rows, payloads worker, OpenAPI schemas.
Règle d'implémentation	DTO à la frontière, mapper vers Command/Entity/Result.
Vérification	Tests de contrat JSON et diff OpenAPI.
Abus interdit	Réutiliser les entités domaine comme payload HTTP.

4.10 Adapter Pattern

Dimension	Convention
Objectif	Isoler les systèmes externes et leurs erreurs.
Utilisation	SQLx, MinIO/S3, filesystem local, PyTorch, ONNX, CoreML.
Règle d'implémentation	L'adapter implémente un port et traduit les erreurs externes.
Vérification	Tests avec dépendances locales réelles : PostgreSQL, MinIO, filesystem.
Abus interdit	Laisser des types SQLx ou boto/torch remonter dans le domaine.

4.11 Factory / Builder Pattern

Dimension	Convention
Objectif	Construire le graphe de dépendances de manière explicite.
Utilisation	AppState, clients DB/storage, repositories, worker runtime.
Règle d'implémentation	Bootstrap unique, config typée, erreurs startup explicites.
Vérification	Test de démarrage avec config manquante ou invalide.
Abus interdit	Service locator global mutable.

4.12 Error Mapping Pattern

Dimension	Convention
Objectif	Garder des erreurs internes riches et des erreurs publiques stables.
Utilisation	DomainError, AppError, RepositoryError, ApiError, WorkerError.
Règle d'implémentation	Mapping contrôlé à chaque frontière. Pas de stack trace côté client.
Vérification	Tests d'erreurs structurées et sûres.
Abus interdit	Retourner des strings libres ou des chemins internes.

4.13 Lightweight Event Log

Dimension	Convention
Objectif	Tracer les étapes importantes sans faire d'event sourcing complet.
Utilisation	JobQueued, JobStarted, JobSucceeded, JobFailed, ModelRegistered, ExportSucceeded.
Règle d'implémentation	Table job_events ou audit_events optionnelle mais append-only.
Vérification	BDD peut vérifier la lisibilité du lifecycle.
Abus interdit	Implémenter une architecture event sourcing complète en MVP.

5. Patterns par module produit

Module	Patterns principaux	Pourquoi
Dataset Management	Use Case, Repository, DTO/Mapper, Value Object	Créer des datasets typés et traçables.
Sample Ingestion	Adapter, Unit of Work, Value Object	Synchroniser fichier stocké et metadata DB.
Annotation	Value Object, Repository	Empêcher les bbox invalides.
Dataset Versioning	Snapshot, Unit of Work, State Machine	Garantir l'immuabilité d'une version.
Training Jobs	State Machine, Queue Adapter, Unit of Work	Éviter le double traitement et les statuts incohérents.
Metrics	Repository, DTO/Mapper	PersistER des mesures typées par epoch.
Model Registry	Repository, State Machine, ArtifactUri	Relier modèle, job, dataset version et artefact.
Inference	Strategy, Adapter, DTO/Mapper	Changer Torch/ONNX/fake sans changer l'API.
Export	Strategy, State Machine, Artifact Store	Support ONNX/CoreML progressivement.

6. State machines obligatoires

Training Job

queued -> running -> succeeded
queued -> running -> failed
queued -> cancelled

Interdit:

running -> queued
succeeded -> running
failed -> succeeded sans nouveau retry job

Model

candidate -> validated -> promoted -> archived
candidate -> archived
validated -> archived
promoted -> archived

Règle:

un modèle ne peut être créé qu'après un training job réussi.

Export

queued -> running -> succeeded
queued -> running -> failed

Règle:

un export doit référencer un modèle existant et conserver une erreur lisible en cas d'échec.

Dataset

draft -> ready -> archived

Dataset mutable: draft

Dataset entraînable: ready

Dataset historique: archived

DatasetVersion: immuable dès création

7. Patterns interdits et anti-patterns

Anti-pattern	Pourquoi c'est dangereux	Remplacement
God service	Cache les flux métier et devient instable	Un use case par opération
SQL dans les handlers	Couple HTTP et persistance	Repository port + adapter SQLx
dict Python brut	Casse au runtime dans le worker	Pydantic strict
serde_json::Value domaine	Aucun invariant	Value objects typés
String status	États impossibles possibles	Enum + transitions
Singleton mutable	État caché et tests pollués	Injection explicite
utils/common/misc	Responsabilité illisible	Nom spécifique par comportement
Framework-driven domain	Le métier dépend du framework	Domaine pur
Mock-only E2E	Ne prouve pas la stack	BDD avec vraie API/DB/storage + fake training

Règle de review : tout anti-pattern introduit dans une PR doit être bloqué, sauf ADR explicite qui documente le compromis.

8. Conventions fichiers/dossiers Rust

La séparation en crates rend les règles de dépendance exécutables. Ce n'est pas une décoration : cela empêche le domaine d'importer accidentellement Axum, SQLx ou MinIO.

```
api/crates/  
  perception_domain/  
    src/  
      ids.rs  
      bbox.rs  
      dataset.rs  
      sample.rs  
      annotation.rs  
      dataset_version.rs  
      training_job.rs  
      model.rs  
      artifact.rs  
      errors.rs  
  
  perception_app/  
    src/  
      ports/  
        dataset_repository.rs  
        sample_repository.rs  
        training_job_repository.rs  
        object_storage.rs  
        inference_engine.rs  
      use_cases/  
        create_dataset.rs  
        upload_sample.rs  
        add_annotation.rs  
        create_dataset_version.rs  
        create_training_job.rs  
        run_inference.rs  
  
  perception_infra/  
    src/  
      postgres/  
        dataset_repository_sqlx.rs  
        sample_repository_sqlx.rs  
        training_job_repository_sqlx.rs  
        model_repository_sqlx.rs  
      storage/  
        s3_object_storage.rs  
        local_object_storage.rs  
      queue/  
        postgres_training_queue.rs  
      config/  
        app_config.rs  
  
  perception_http/  
    src/  
      routes/  
        dto/  
          mappers/  
            error_response.rs  
            openapi.rs  
  
  perception_api/  
    src/  
      main.rs  
      bootstrap.rs
```

Règles Rust

- Un fichier = une responsabilité claire.
- Les use cases sont nommés par verbe + nom : create_dataset.rs, upload_sample.rs.
- Les adapters SQLx finissent par _sqlx.rs.
- Les modules fourre-tout utils.rs, helpers.rs, misc.rs sont interdits.
- Le domaine n'importe jamais Axum, SQLx, MinIO, PyTorch ou OpenAPI.

9. Conventions fichiers/dossiers Python worker

Le worker Python doit être un vrai composant applicatif typé, pas une collection de scripts. PyTorch reste isolé dans les adaptors training/inference.

```
worker/perception_worker/  
  domain/  
    training_job.py  
    training_result.py  
    metrics.py  
    model_artifact.py  
  
  contracts/  
    job_payloads.py  
    inference_payloads.py  
    export_payloads.py  
  
  app/  
    process_training_job.py  
    process_export_job.py  
    run_inference.py  
  
  ports/  
    job_repository.py  
    artifact_store.py  
    trainer.py  
    inference_engine.py  
    exporter.py  
  
  adapters/  
    db/  
      postgres_job_repository.py  
    storage/  
      s3_artifact_store.py  
      local_artifact_store.py  
    training/  
      fake_trainer.py  
      tiny_trainer.py  
      yolo_trainer.py  
    inference/  
      fake_inference_engine.py  
      torch_inference_engine.py  
      onnx_inference_engine.py  
    export/  
      onnx_exporter.py  
      coreml_exporter.py  
  
  entrypoints/  
    worker.py  
    inference_server.py
```

Règles Python

- Les contracts utilisent Pydantic strict.
- Les ports utilisent Protocol quand il y a plusieurs implémentations possibles.
- Les entrypoints câblent les dépendances mais ne contiennent pas de logique métier.
- torch est interdit hors adapters/training et adapters/inference.
- Pas de dict brut pour les payloads de job, export ou inference.

10. Conventions QA, contracts, docs et scripts

QA

```
qa/  
  features/  
    dataset_management.feature  
    sample_ingestion.feature  
    training_jobs.feature  
    inference_api.feature  
    end_to_end_pipeline.feature  
  steps/  
    test_dataset_steps.py  
    test_sample_steps.py  
    test_training_steps.py  
    test_inference_steps.py  
  support/  
    api_client.py  
    db.py  
    storage.py  
    waiters.py  
    assertions.py  
  fixtures/  
    images/  
    payloads/  
    annotations/
```

Contracts

```
contracts/  
  openapi.json  
  training_job.schema.json  
  inference_response.schema.json  
  metrics.schema.json
```

Docs

```
docs/  
  architecture.md  
  architecture-conventions.md  
  design-patterns.md  
  qa-bdd.md  
  adr/  
    0001-use-hexagonal-architecture.md  
    0002-use-postgresql-backed-queue-for-mvp.md  
    0003-use-strategy-for-training-and-inference-modes.md
```

Scripts

```
scripts/  
  test_all.sh  
  test_bdd.sh  
  reset_test_env.sh  
  seed_demo_dataset.sh  
  export_openapi.sh
```

11. Conventions de nommage

Élément	Convention	Exemple
Rust crate	snake_case avec prefix perception_	perception_domain
Rust module	snake_case, responsabilité précise	normalized_bbox.rs
Use case	verbe_nom	create_training_job.rs
Repository trait	Nom métier + Repository	DatasetRepository
SQLx adapter	nom + _sqlx.rs	dataset_repository_sqlx.rs
Python module	snake_case précis	dataset_materializer.py
BDD feature	module produit	training_jobs.feature
API endpoint	ressource plurielle, kebab-case	/training-jobs/{id}
JSON	snake_case	dataset_version_id
ADR	numéro + slug	0004-use-pydantic-strict.md

Règle des fichiers interdits

Interdit	Remplacement attendu
utils.rs / utils.py	checksum_calculator.rs, bbox_validation.rs, metrics_mapper.py
helpers.rs / helpers.py	api_client.py, storage_assertions.py, dataset_materializer.py
common/	domain/, contracts/, support/ ou fixtures/ selon la responsabilité
manager.rs	model_registry.rs, training_job_repository.rs, artifact_store.py
service.rs	create_dataset.rs, run_inference.rs, s3_object_storage.rs

12. Exemples Rust

Newtype ID

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct DatasetId(uuid::Uuid);

#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct ModelId(uuid::Uuid);
```

Value Object avec invariant

```
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct NormalizedBbox {
    x: f32,
    y: f32,
    width: f32,
    height: f32,
}

impl NormalizedBbox {
    pub fn new(x: f32, y: f32, width: f32, height: f32) -> Result<Self, BboxError> {
        if x < 0.0 || y < 0.0 || width <= 0.0 || height <= 0.0 {
            return Err(BboxError::InvalidCoordinates);
        }
        if x + width > 1.0 || y + height > 1.0 {
            return Err(BboxError::OutOfBounds);
        }
        Ok(Self { x, y, width, height })
    }
}
```

Repository port

```
#[async_trait::async_trait]
pub trait DatasetRepository {
    async fn insert(&self, dataset: Dataset) -> Result<Dataset, RepositoryError>;
    async fn find_by_id(&self, id: DatasetId) -> Result<Option<Dataset>, RepositoryError>;
}
```

Use case

```
pub struct CreateDatasetUseCase<R>
where
    R: DatasetRepository,
{
    repository: R,
}

impl<R> CreateDatasetUseCase<R>
where
    R: DatasetRepository,
{
    pub async fn execute(&self, command: CreateDatasetCommand) -> Result<Dataset, AppError> {
        let dataset = Dataset::new(command.name, command.task_type, command.classes)?;
        self.repository.insert(dataset).await.map_err(AppError::from)
    }
}
```

State transition

```
impl TrainingJob {
    pub fn start(&mut self) -> Result<(), TrainingJobError> {
        match self.status {
            TrainingJobStatus::Queued => {
                self.status = TrainingJobStatus::Running;
                Ok(())
            }
            _ => Err(TrainingJobError::InvalidTransition),
        }
    }
}
```

13. Exemples Python worker

Pydantic strict contract

```
from pydantic import BaseModel, ConfigDict, PositiveFloat, PositiveInt
```

```
class TrainingHyperparameters(BaseModel):
    model_config = ConfigDict(strict=True, extra="forbid", frozen=True)

    epochs: PositiveInt
    batch_size: PositiveInt
    image_size: PositiveInt
    learning_rate: PositiveFloat
```

Protocol Strategy

```
from typing import Protocol
from perception_worker.domain.training_job import TrainingJob
from perception_worker.domain.training_result import TrainingResult
```

```
class Trainer(Protocol):
    def train(self, job: TrainingJob) -> TrainingResult:
        ...
```

```
class FakeTrainer:
    def train(self, job: TrainingJob) -> TrainingResult:
        return TrainingResult.fake_success(job_id=job.id)
```

FakeTrainer pour CI

```
class FakeTrainer:
    def train(self, job: TrainingJob) -> TrainingResult:
        return TrainingResult(
            job_id=job.id,
            metrics=[
                TrainingMetric(epoch=1, train_loss=0.82, val_loss=0.91, map50=0.61),
                TrainingMetric(epoch=2, train_loss=0.55, val_loss=0.62, map50=0.74),
            ],
            artifact_uri=f"s3://perceptionlab-test/models/{job.id}.pt",
        )
```

Application service

```
class ProcessTrainingJob:
    def __init__(
        self,
        jobs: JobRepository,
        trainer: Trainer,
        artifacts: ArtifactStore,
    ) -> None:
        self._jobs = jobs
        self._trainer = trainer
        self._artifacts = artifacts

    def execute(self, job_id: TrainingJobId) -> None:
        job = self._jobs.mark_running(job_id)
        try:
            result = self._trainer.train(job)
            self._artifacts.ensure_exists(result.artifact_uri)
            self._jobs.mark_succeeded(job_id, result)
        except Exception as exc:
            self._jobs.mark_failed(job_id, str(exc))
```

14. Tests attendus par pattern

Pattern	Tests obligatoires
Newtype	Parsing, sérialisation, refus des IDs invalides.
Value Object	Constructeurs valides/invalides, bornes et invariants.
State Machine	Transitions autorisées et transitions interdites.
Use Case	Happy path, erreur validation, erreur repository/storage.
Repository	Tests intégration PostgreSQL avec migrations.
Unit of Work	Rollback si échec et absence de lignes orphelines.
Strategy	Test de contrat commun pour fake/tiny/real.
DTO + Mapper	Validation payload, forme JSON, diff OpenAPI.
Adapter	Test avec dépendance locale réelle : MinIO, PostgreSQL, filesystem.

Mapping tests vers dossiers

Type de test	Dossier	But
Rust unit	api/crates/*/tests ou src tests	Value objects, state machines, use cases.
Rust integration	api/tests/	SQLx, routes Axum, storage adapters.
Python unit	worker/tests/	Pydantic, strategies, worker app.
BDD	qa/features/	Produit complet via API, DB, storage, worker.

15. Règles d'import et dépendances

Rust:

```
perception_domain -> aucune crate projet
perception_app    -> perception_domain
perception_infra  -> perception_app + perception_domain
perception_http   -> perception_app + perception_domain
perception_api    -> perception_http + perception_infra
```

Python:

```
entrypoints -> app -> ports/domain/contracts
adapters    -> ports/domain/contracts
training adapters -> torch autorisé
repositories -> torch interdit
```

Contrôle PR

- Un import interdit bloque la PR.
- Une dépendance technique dans le domaine bloque la PR.
- Un fichier vague bloque la PR ou impose un renommage.
- Une exception doit être documentée dans un ADR.

16. Checklist de review architecture

Check	Question de review
Layering	La dépendance suit-elle http/infra -> app -> domain ?
Domain	Les concepts métier ont-ils des types explicites ?
Handlers	Les handlers sont-ils sans SQL et sans orchestration métier lourde ?
Repositories	La persistance est-elle derrière un port ?
Transactions	Les opérations multi-écritures sont-elles atomiques ?
Worker	Les payloads sont-ils Pydantic strict et torch isolé ?
Contracts	OpenAPI/schema est-il à jour si le JSON public change ?
Files	Les noms de fichiers/dossiers sont-ils spécifiques ?
Tests	Le pattern introduit a-t-il le bon niveau de test ?
ADR	La décision mérite-t-elle un ADR ?

17. Roadmap d'implémentation

Étape	Livrable	Validation
1	Créer docs/design-patterns.md et ADR skeleton	Source de vérité architecture disponible
2	Créer les crates Rust	cargo check vérifie la séparation
3	Implémenter IDs, statuses, bbox	Tests unitaires domaine
4	Créer ports + premier use case	Test use case avec repository fake
5	Créer adapter SQLx	Test intégration PostgreSQL
6	Créer DTO HTTP + mapper	API test POST /datasets
7	Créer contracts worker + Trainer Protocol	Pyright strict + pytest
8	Créer FakeTrainer	BDD training lifecycle vert
9	Ajouter règles review fichiers	PR checklist bloque utils/common/misc
10	Exécuter make ci	Tous les gates passent

Première victoire attendue

```
make ci
```

```
docker compose -f docker-compose.test.yml up -d  
pytest qa -m "p0"
```

18. Synthèse

PerceptionLab doit être un projet où l'architecture empêche les erreurs banales. Le code doit rendre les mauvais chemins difficiles : impossible de créer une bbox invalide, difficile d'écrire du SQL dans un handler, évident de choisir un adapter plutôt qu'un raccourci technique.

- Chaque opération produit est un use case.
- Chaque système externe est un adapter.
- Chaque valeur critique est un type.
- Chaque lifecycle mutable est une state machine.
- Chaque contrat public a un DTO et une représentation OpenAPI/schema.
- Chaque nom de fichier ou dossier doit annoncer sa responsabilité.
- Chaque exception aux règles doit avoir un ADR.

Phrase de défense en entretien : PerceptionLab est une plateforme ML infrastructure domain-driven : Rust orchestre les flux typés, Python exécute les stratégies ML, PostgreSQL protège la cohérence, les adapters isolent les systèmes externes et la BDD prouve le comportement end-to-end.